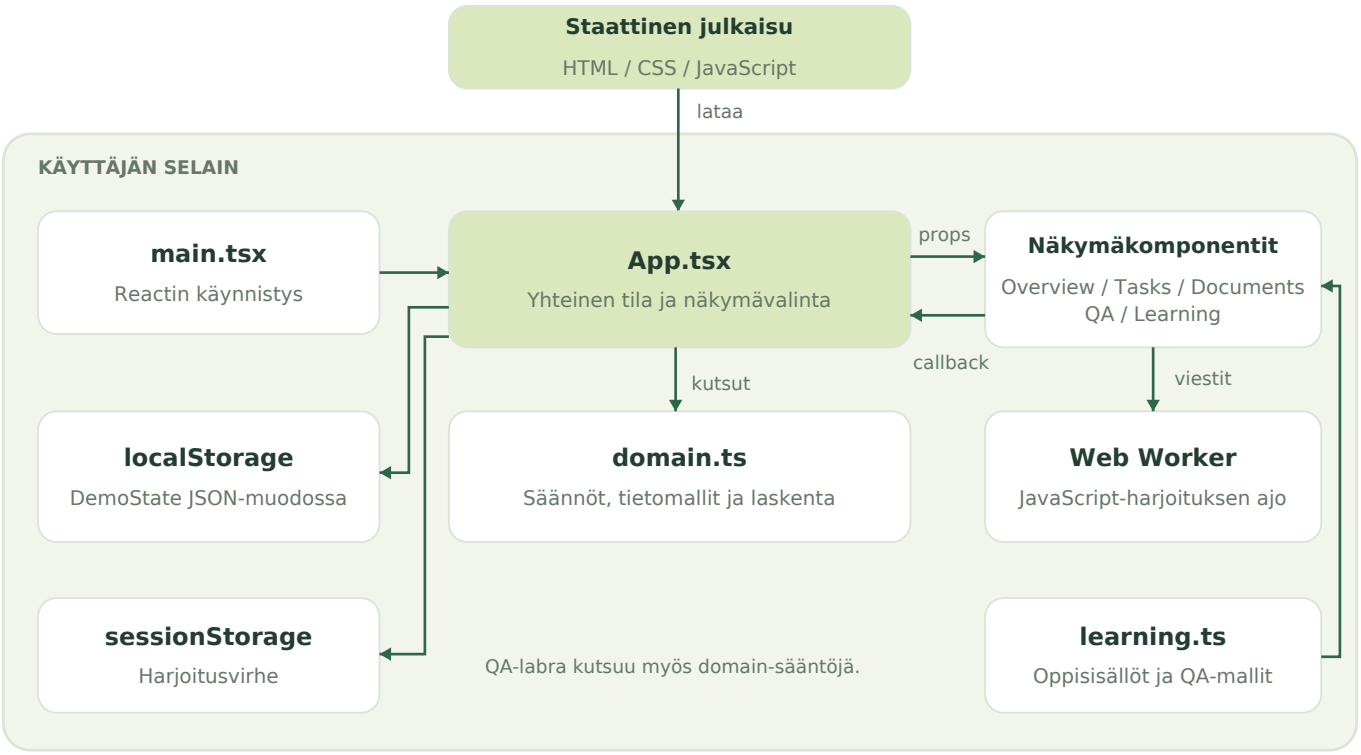
Sovellus rakentuu selaimessa

React/TypeScript-harjoitussovelluksen rakenne, tiedonkulku ja testaus. Kuvaus perustuu julkaistun version 1 tarkistettuun lähdekoodiin.



Perusratkaisu
Yhden sivun sovellus (SPA). Käyttöliittymä, liiketoimintasäännöt ja harjoitusdata toimivat selaimessa. Oma backend-API, palvelintietokanta ja dokumenttien tiedostopalvelu puuttuvat.

Teknologiat ja niiden tehtävät

Teknologia	Tehtävä
JavaScript ja TypeScript	JavaScript ajetaan selaimessa. TypeScript kuvaa tietotyypit ja tarkistaa sopimukset ennen käännöstä.
React / TSX	Komponentit, tapahtumat, useState-tila ja näkymän päivitys.
Vite	Kehityspalvelin sekä staattinen tuotantokäännös dist-hakemistoon.
Vitest / Playwright	Liiketoimintafunktioiden testit ja käyttäjän työnkulkujen selaintestit.

Lähdecommit: 87143b7bea1b3badf96391f87b656842d2b45bf7. React 19-, TypeScript 5.9- ja Vite 7 -päälinjat; tarkat versiot määräytyvät lukitustiedostosta. Tämä on kuvitteellinen harjoitus, ei Admicomin sisäinen arkkitehtuuri.

Komponenteilla on selkeät vastuut

App omistaa yhteisen tilan. Näkymät näyttävät dataa ja palauttavat muutosesitykset callbackeilla. Domain-moduuli ratkaisee, hyväksytäänkö muutos.

Moduuli	Vastuu
`main.tsx`	Käynnistää Reactin root-elementtiin, ottaa StrictMode-tilan käyttöön ja lataa CSS:n.
`App.tsx`	DemoState, projektivalinta, näkymävalinta, harjoitusrooli ja virhevalinta. Selaintallennuksen effectit ja muutoskäsittelijät.
`Overview.tsx`	Projektin mittarit, tehtävien valmistuminen ja budjetti. Arvot johdetaan nykyisestä datasta.
`Tasks.tsx`	Tehtävätaulu, haku, prioriteettisuodatus, tilavalinta ja TaskDialog-lomake.
`Documents.tsx`	Dokumenttikortit, versiohistoria, roolinvalinta ja uuden version dialogi.
`domain.ts`	Tietomallit, demoaineisto, validointi, riippuvuudet, versiosäännöt, budjettilaskenta ja readState.
`QA.tsx`	Mallipohjaiset testiluonnokset, AI-kehotteet, arviointilista ja runDomainChecks-tulokset.
`Learning.tsx`	Oppimispolku, JavaScriptin suoritus sekä React- ja TypeScript-harjoitusten käyttöliittymät.
`learning.ts`	Oppituntien sisältö ja jsStarter; lisäksi TestCase, cases ja makePrompt QA-labraa varten.
`ui.tsx` `styles.css`	Yhteiset dialogit, koodilohkot, avatarit, piirros ja responsiivinen ulkoasu.

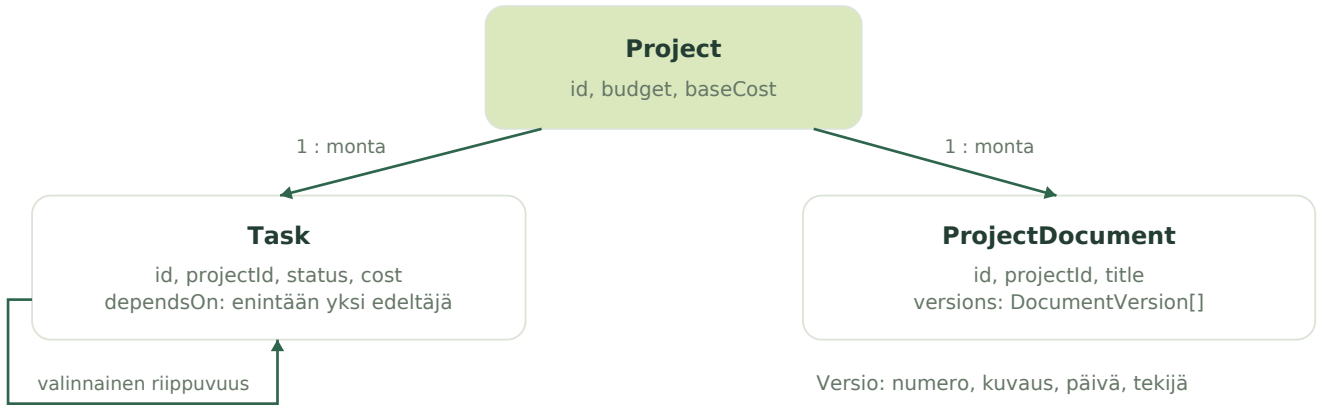
Tila kulkee alas, tapahtuma takaisin ylös

Esimerkiksi Tasks saa tehtävät propsina. Kun käyttäjä muuttaa tilaa, komponentti kutsuu `onStatus`-callbackia. App kutsuu `changeTaskStatus`-funktiota ja päivittää vasta hyväksytyn tuloksen yhteiseen tilaan. Muutosfunktiot palauttavat uuden taulukon tai olion.

Paikallinen tila on näkymässä
Haku, lomakkeen kentät, avoin versioikkuna ja QA-luonnos eivät kuulu pysyvään DemoStateen. Näkymästä poistuminen nollaa sen komponentin paikallisen tilan. Tasks ja Documents alustuvat uudelleen myös projektin vaihtuessa.

Data ja tallennuksen elinkaari

Suhteet ovat TypeScript-olioiden suhteita. Kaavio ei esitä toteutettua tietokantamallia.



Tieto	Paikka ja elinkaari
Tehtävät, versiot ja harjoittelumerkinnät	DemoState: `schema, tasks, documents, completedLessons`. App-tila tallentuu localStorage-avaimeen `rakenna-qa-lab-v1`.
Projektien perustiedot	projects-vakio domain.ts-moduulissa. Tietoja ei tallenneta osana DemoStatea.
Harjoitusvirhe	App.fault ja sessionStorage-avain `rakenna-fault`. Säilyy latauksessa kyseisessä välilehti-istunnossa.
Projekti, näkymä ja rooli	Vain App-komponentin tilassa. Latauksessa oletukset ovat p1, overview ja manager.
Näkymäkohtainen tieto	Haku, lomakkeet ja testitulokset säilyvät vain kyseisen komponentin elinkaaren ajan.

Käynnistys ja muutokset

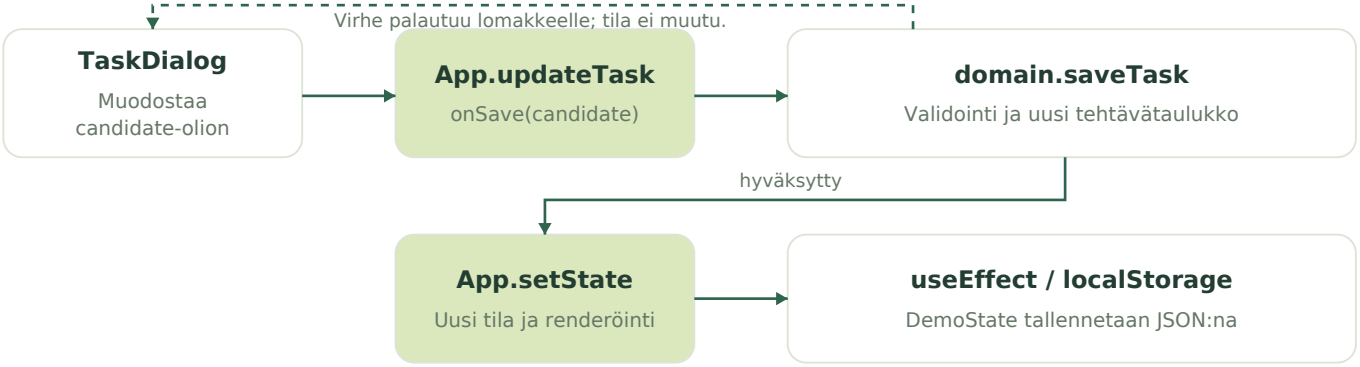
`readState()` lukee JSON:n ja tarkistaa skeemaversion sekä päätason taulukot. Puuttuva tai virheellinen tallennus vaihtuu alkuperäiseen demoaineistoon. Tarkistus ei validoi kaikkia sisäisiä kenttiä.

Tilamuutoksen jälkeen useEffect kirjoittaa koko DemoState-olion. Tallennusvirheessä sovellus jatkaa muistissa ja näyttää asiasta tiedon. Tiedot eivät synkronoidu laitteiden tai originien välillä; välilehtien muutoksille ei ole yhdistämistä.

Demon täsmälliset rajat
dependsOn on valinnainen yhden edeltäjän tunniste. Versiot ovat dokumentin sisäisiä olioita. Vastuuhenkilö ja tekijä ovat nimimerkkijonoja. Harjoituksen päivä on koodissa 7.11.2026.

Muutos kulkee sääntöjen kautta

Esimerkki: käyttäjä tallentaa tehtävän. Validointi ja tilapäivitys ovat eri vaiheita kuin pysyvä selaintallennus.



Tallennusvirhe ei peru jo hyväksyttyä muistissa olevaa muutosta.

Funktio	Sääntö
`validateTask`	Nimi pakollinen, enintään 120 merkkiä. Äärellinen kustannus vähintään nolla. Oikea ISO-päivämäärä.
`saveTask`	Lisää tai korvaa tehtävän. Estää riippuvuuskehät ja toisen projektin edeltäjän; tarkistaa valmiin tehtävän riippuvuuden.
`changeTaskStatus`	Jatkotehtävä valmistuu vasta edeltäjän jälkeen. Edeltäjää ei avata valmiin jatkotehtävän alta.
`addDocumentVersion`	Manager-rooli ja muutoskuvaus vaaditaan. Numero on edellinen maksimi + 1. Aiemmat versiot säilyvät.
`budgetSummary`	Sidottu arvio = pohjakustannukset + saman projektin tehtävien arviot. Jäljellä = budjetti - sidottu arvio.

Mitä onnistuminen tarkoittaa?

Validointipoikkeus estää yhteisen tilan muutoksen ja palaa lomakkeen tai näkymän virheilmoitukseksi. Hyväksytty muutos näkyy heti React-tilassa. Onnistumisilmoitus ei yksin todista pysyvää tallennusta, koska localStorage-kirjoitus seuraa erillisessä effectissä.

Testauksen kannalta olennaista
Varmista sekä muuttunut käyttöliittymä että tiedon säilyminen latauksen jälkeen. Budjetin ylitys on varoitus, ei tallennuksen esto. Dokumenttiversio tallentaa metatietoja; esikatselun piirros ei ole ladattu PDF-tiedosto.

Testaus ja AI-harjoittelu

QA-labra, yksikkötestit ja selaintestit käyttävät eri suoritustapoja ja osoittavat eri asioita.

Taso	Toteutus ja tarkoitus
QA-labran 6 tarkistusta	`runDomainChecks(fault)` käyttää uutta vakioaineistoa jokaisessa ajossa. Testaa samoja funktioita kuin sovellus, ei DOM:ia.
26 Vitest-tapausta	`tests/unit/domain.test.ts`. Validointi, päivämäärät, riippuvuudet, roolit, budjetti ja harjoitusvirheet.
7 Playwright-testiä	`tests/e2e/project.spec.ts`. Käyttäjän toimet, näkyvä lopputulos, latauksen yli säilyminen ja projektirajaus.

Määrät kuvaavat julkaistun lähdeversion testisisältöä. Tässä dokumentointityössä ei tehty uutta testiajoa. Käyttäjän omat lisätestit voivat muuttaa määriä.

Harjoitusvirhe on yhteinen sovellustila

`Fault` on none, dependency, version tai permission. Ilman virhettä labran tulos on 6/6; kukin kolmesta virheestä tuottaa odotetusti yhden epäonnistumisen eli 5/6. Sama valinta vaikuttaa myös tehtävien ja dokumenttien toimintaan. Virheen poistaminen ei peru jo tallennettua demomuutosta.

AI-osio tuottaa mallin ja kehotteen

`learning.ts` sisältää testimallit ja makePrompt-funktion. QA näyttää valitun mallin sekä kopioitavan AI-kehotteen. Lisäkonteksti siirtyy kehotteeseen, ei automaattisesti mallikoodiin. Käyttäjä käyttää ulkoista AI-työkalua ja arvioi tuotoksen. Sovellus ei kutsu kielimalli-API:a. Ladatun testin suoritus tapahtuu erikseen Playwrightilla.

JavaScript-harjoitus käyttää omaa workeria



Worker suljetaan vastauksen jälkeen tai 1,5 sekunnissa. Se ei ole täydellinen turvaraja.

Worker saa viestillä tehtävädatan kopion. Se suorittaa oman harjoituskoodin ja palauttaa tuloksen tai virheen. Tämä erottaa suorituksen käyttöliittymäsäikeestä; se ei muodosta täydellistä eristystä epäluotettavalle koodille.

Kaksi erilaista käyttöoikeusrajaa
Julkaisupalvelun ChatGPT-kirjautuminen rajaa yksityiselle sivustolle pääsyä. Sovelluksen manager/viewer on harjoitusrooli selaimen sisällä. Todellinen SaaS-käyttöoikeus olisi tarkistettava myös palvelimella.

Kehitys, julkaisu ja version raja

Kehitystyökalut tuottavat staattisen sovelluksen. Kuvaus erottaa nykyiset ominaisuudet ja ehdotetut jatkomuutokset.

Vaihe	Toteutus
Paikallinen kehitys	`npm run dev` käynnistää Viten. Node.js on kehitys- ja käännöstyökalu, ei liiketoimintapalvelin.
Tyypit ja testit	`npm run typecheck`, `npm test` ja `npm run test:e2e`. Playwright käynnistää Viten porttiin 4173 ja käyttää erillisiä selainkonteksteja.
Tuotantokäännös	`npm run build`: lähdekoodi-ZIP, tsc --noEmit ja vite build. Julkaisuun jaetaan dist-hakemisto.
CI / quality.yml	Node 22 / Ubuntu: npm ci, tyypit, yksikkötestit, Chromium, E2E ja build. Epäonnistuessa raportti 7 päiväksi. Yhdistämisen esto vaatii pakollisen status checkin. Työnkulku ei itse julkaise.

Perustellut jatkokehityssuunnat

Nykyinen rakenne	Seuraava vastuu
Paikallinen tallennus	Storage-adapteri, syvä skeemavalidointi ja myöhemmin keskitetty API/tietokanta.
Harjoitusroolit ja dokumentin metatiedot	Palvelimen käyttäjä- ja asiakasrajaukset sekä varsinainen tiedostovarasto.
Yhteinen learning.ts	Erota oppituntisisällöt ja QA-mallit omiin moduuleihin. Tämä pienentää harjoitusmuutosten vaikutusta labraan.

Erilliset harjoitustiedostot vaativat yhteensovituksen

Lisäksi luetuista 7.11.2026 tallennetuista erillistiedostoista learning.ts sisältää oppitunnit ja jsStarterin. Siitä puuttuvat julkaistun QA.tsx:n tarvitsemat `TestCase, cases, makePrompt`-viennit. Sen suora korvaaminen perusprojektiin rikkoisi import-sopimuksen. Erillisessä Learning.tsx:ssä on yksi aktiivinen avoimien tehtävien suodatin ja aiempi toteutus kommentoituna.

Kokonaiskaaviot kuvaavat tarkistettua julkaistua perusversiota. Käyttäjän koko Windows-projektia tai sen uusimpia testituloksia ei ole tässä tarkistettu. Dokumentointi ei muuttanut sovelluskoodia.

Haastattelussa rakenne tiivistyy näin
"App omistaa tilan, näkymät käyttävät propseja ja callbackeja, ja domain-moduuli ratkaisee liiketoimintasäännöt. Säännöt ovat suoraan testattavia, Playwright varmistaa käyttäjän polut ja AI tukee testien luonnostelua. Tuotantopalveluun lisäisin palvelimen käyttöoikeudet ja keskitetyn tallennuksen."